



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

AI-Enhanced Smart Waste Management System

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA1703-Artificial Intelligence

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

(AI&DS)

Submitted by

Pujitha K (192524149)

Under the Supervision of

Dr. Jeena R

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “AI-Enhanced Smart Waste Management System” has been carried out by Pujitha K (192524149) under the supervision of Dr. R. Jeena and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech (AI&DS) program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

**Dr.Krishna Kumar Kathiresan
Professor
CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.**

COURSE FACULTY

**Dr.Jeena R
Professor
CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.**

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, Pujitha K (192524149) of the B.tech (AI&DS), SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “AI-Enhanced Smart Waste Management System” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Students with Names

Pujitha K (192524149)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Pujitha K 192524149	System design and data collection	System architecture and implementati on	System testing and result analysis	Abstract, Introduction and Design chapters	35%
Hashini S 192524035	AI model and prediction	AI model development and integration	Model testing and accuracy evaluation	Literature Review and Methodology	35%
Sandhiya 192511132	Monitoring and documentation	Dashboard and monitoring module	Performance testing and validation	Results, Conclusion and References	30%
Total					100%

ABSTRACT

Effective waste management is a major engineering challenge due to increasing waste generation, improper disposal, overflowing garbage bins, and inefficient waste collection practices. Conventional waste collection systems generally operate according to fixed schedules, which can result in unnecessary collection trips when bins are partially empty and delayed collection when bins become full. These problems can increase operational costs, fuel consumption, environmental pollution, and public health risks. Therefore, an intelligent and automated waste management system is required to monitor waste conditions and support timely collection.

The proposed AI-Enhanced Smart Waste Management System integrates Artificial Intelligence, sensors, and data-driven analysis to improve the efficiency of waste collection and monitoring. The system continuously monitors the fill level of waste bins and analyses the collected data to determine when waste collection is required. AI-based techniques can be used to predict bin-filling patterns and identify bins that are likely to reach their capacity soon. This enables waste collection to be performed based on actual waste conditions rather than relying only on predefined schedules.

The methodology involves collecting real-time data from smart waste bins using sensors, transmitting the data to a central monitoring platform, preprocessing the collected information, and applying AI-based prediction techniques. The system analyses parameters such as bin fill level, time, and historical filling patterns to estimate future waste accumulation. Based on predefined thresholds and prediction results, the system generates alerts and prioritises bins requiring collection. A monitoring dashboard can also provide information about bin status and collection requirements.

The major results are expected to demonstrate improved waste collection efficiency, reduced unnecessary collection trips, better utilisation of collection vehicles, and timely identification of overflowing bins. The predictive capability of the system can further support proactive waste collection and improve operational planning.

In conclusion, the AI-Enhanced Smart Waste Management System provides an intelligent, efficient, and sustainable approach to modern waste management. By combining real-time monitoring with AI-based prediction, the proposed system can reduce collection inefficiencies, minimise environmental impact, and support cleaner and smarter communities.

Keywords: Artificial Intelligence, Smart Waste Management, IoT Sensors, Waste Prediction, Real-Time Monitoring, Sustainable Waste Management

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
lii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	10-14
2	CHAPTER 2: Literature Review	15-20
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	21-39
4	CHAPTER 4: Implementation, Testing & Results, Project Management	40-46
5	CHAPTER 5: Conclusion & Reflection	47-50
	References	49-50
	Appendices	51-54

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.	Smart Waste Management System Architecture	25
2.	IoT-Based Smart Waste Collection System	26
3.	Specific roles of sensors or AWS services	26

LIST OF TABLES

Table No.	Table Name	Page No.
1.	Comparitive analysis	20
2.	Stakeholder / User Requirements for the AI-Enhanced Smart Waste Management System	22
3.	Functional & Non-Functional Requirements	22
4.	Design Specifications	23
5.	Sustainability, Ethics, Safety and Security	28

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
NLP	Natural Language Processing	1
FOL	First Order Logic	3
EBL	Explanation Based Learning	4

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Waste generation is increasing rapidly due to population growth, urbanisation, commercial activities, and changes in consumption patterns. Efficient collection and management of municipal waste has therefore become an important engineering and environmental challenge. Conventional waste collection systems commonly depend on fixed schedules, where collection vehicles visit designated locations at predetermined times. However, the actual amount of waste in a bin can vary significantly depending on location, day, population, and usage patterns.

An AI-Enhanced Smart Waste Management System aims to overcome these limitations by combining Internet of Things (IoT) sensors, Artificial Intelligence (AI), data processing, and real-time monitoring. Sensors installed in waste bins can measure parameters such as waste fill level. The collected data can be transmitted to a central monitoring system, where AI-based techniques can analyse current and historical waste patterns.

The system can identify bins that are approaching their capacity and predict which bins may require collection in the near future. Instead of collecting waste only according to a fixed schedule, collection can be planned according to actual and predicted waste conditions. This approach can improve resource utilisation, reduce unnecessary collection trips, and help prevent overflowing bins.

1.2 Need for the Project

Why the Problem Needs to Be Addressed

Traditional waste collection may result in two major problems:

- **Under-utilised collection:** Vehicles visit bins that are not yet full.
- **Delayed collection:** Bins become full before the scheduled collection time.

Both situations reduce the efficiency of waste management. Overflowing waste can also cause unpleasant odours, attract insects and animals, and contribute to environmental pollution.

Who Is Affected by the Problem

The problem affects:

- Residents and local communities
- Municipal authorities
- Waste collection workers
- Waste management companies
- Collection vehicle operators
- The surrounding environment

Existing Limitations

Existing conventional systems have several limitations:

1. Dependence on fixed collection schedules.
2. Lack of continuous monitoring of bin conditions.
3. Limited ability to predict future waste accumulation.
4. Unnecessary trips by collection vehicles.
5. Difficulty in prioritising bins that require immediate collection.
6. Increased fuel consumption and operational costs.

Engineering Significance

The project has engineering significance because it combines sensing, communication, data analysis, AI prediction, and decision-making into a single system. It demonstrates how intelligent technologies can be applied to optimise a real-world infrastructure problem while improving efficiency and sustainability.

1.3 Problem Statement

Conventional waste collection systems operate primarily using fixed schedules and manual monitoring, making it difficult to determine the actual and future waste conditions of individual bins. The engineering problem is to design and develop an intelligent waste management system capable of monitoring bin fill levels in real time, analysing waste accumulation patterns, predicting collection requirements, and prioritising bins for collection while considering system accuracy, response time, operational efficiency, sensor limitations, and resource constraints.

The system must balance multiple requirements, including accurate prediction, timely collection, reduced unnecessary vehicle trips, prevention of bin overflow, and efficient utilisation of collection resources.

1.4 Project Aim

The aim of this project is to design and develop an AI-enhanced smart waste management system that uses real-time monitoring and AI-based prediction to optimise waste collection, reduce unnecessary collection activities, and prevent overflowing waste bins.

1.5 Project Objectives

The main objectives of the project are:

1. To design a smart waste management architecture integrating IoT-based monitoring and AI techniques.
2. To develop a system for monitoring waste-bin fill levels in real time.
3. To collect and process waste accumulation data for analysis and prediction.
4. To implement an AI-based model for predicting future waste accumulation and collection requirements.
5. To develop a monitoring mechanism that identifies and prioritises bins requiring collection.
6. To test the system under different waste accumulation conditions.
7. To evaluate the system based on prediction accuracy, response time, collection efficiency, and reduction in unnecessary collection trips.

1.6 Scope

Included

The project includes:

- Smart waste-bin monitoring.
- Measurement of waste fill level using sensors or simulated sensor data.
- Real-time data collection and processing.
- Storage and analysis of waste-related data.

- AI-based waste accumulation prediction.
- Identification of bins approaching capacity.
- Generation of collection alerts.
- Prioritisation of bins requiring collection.
- Monitoring through a software dashboard or interface.
- Evaluation of system performance using suitable metrics.

Excluded

The project does not include:

- Physical deployment of a city-wide waste collection infrastructure.
- Complete autonomous control of waste collection vehicles.
- Physical transportation of waste.
- Waste treatment or recycling operations.
- Large-scale municipal implementation.
- Handling of hazardous or biomedical waste.

Assumptions

The system assumes that:

- Sensors provide sufficiently reliable fill-level measurements.
- Data can be transmitted to the monitoring platform.
- Historical waste data is available for AI prediction.
- Each monitored bin has a unique identification.
- A predefined threshold can be used to identify critical fill levels.

Boundary Conditions

The system operates within the following boundaries:

- Monitoring is limited to the selected smart bins.
- Prediction accuracy depends on the quality and quantity of available data.
- Sensor errors or communication failures may affect system performance.
- AI predictions are intended to support collection decisions rather than completely replace human supervision.

1.7 Expected Engineering Outcome

The expected engineering outcome is a functional prototype/software-based intelligent waste management system that integrates real-time waste monitoring with AI-based prediction.

The final system is expected to provide:

- Real-time bin-status monitoring.
- Waste accumulation prediction.
- Identification of bins nearing capacity.
- Automatic collection alerts.
- Priority-based collection recommendations.
- A monitoring dashboard for decision-making.

Scope

Clearly state:

- What is included
- What is excluded
- Assumptions
- Boundary conditions

1.2 Expected Engineering Outcome

Define the intended:

- System
- Product
- Process
- Prototype
- Algorithm
- Model
- Device
- Infrastructure solution

as applicable.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

A review of existing literature and technologies was carried out to understand current approaches to smart waste management and identify limitations that can be addressed by the proposed system. Relevant research papers, technical articles, existing smart-bin systems, IoT-based waste monitoring technologies, and AI-based waste prediction methods were considered.

The review focused on the following areas:

- IoT-based smart waste-bin monitoring
- Waste-bin fill-level detection
- Real-time waste monitoring systems
- AI and machine-learning-based waste prediction
- Smart waste collection and route optimisation
- Sensor-based waste management
- Commercial smart waste-management solutions

The existing approaches were compared based on monitoring capability, prediction capability, collection efficiency, real-time operation, scalability, cost, and limitations. The findings from the review were used to identify the engineering gap and define the requirements of the proposed system.

2.2 Review of Existing Work

1. Conventional Scheduled Waste Collection

Traditional waste management generally uses predetermined collection schedules. Collection vehicles visit specific locations at fixed times, regardless of the actual amount of waste present in individual bins.

Advantages:

- Simple to implement.
- Easy to schedule.

- Does not require complex technology.

Limitations:

- Bins may be collected when they are only partially filled.
- Full bins may remain uncollected until the next scheduled visit.
- Vehicle routes may be inefficient.
- Fuel and manpower can be unnecessarily consumed.

This approach provides the basic foundation for waste collection but does not provide intelligent, condition-based decision-making.

2. IoT-Based Smart Waste-Bin Monitoring

IoT-based systems use sensors such as ultrasonic sensors to measure the amount of waste inside a bin. The measured information can be transmitted through communication networks to a central monitoring platform.

Advantages:

- Enables real-time monitoring.
- Reduces dependence on manual inspection.
- Provides information about individual bin conditions.
- Can generate alerts when bins approach their capacity.

Limitations:

- Mainly focuses on the current fill level.
- Does not necessarily predict future waste accumulation.
- Sensor errors and communication failures can affect reliability.
- Large-scale deployment can increase infrastructure and maintenance costs.

IoT monitoring is therefore useful for the proposed project as the data acquisition layer.

3. AI/Machine-Learning-Based Waste Prediction

Machine-learning techniques can analyse historical waste-generation data to predict future waste accumulation. Parameters such as previous fill levels, time, day, and usage patterns can be used to estimate when a bin may reach its capacity.

Advantages:

- Enables predictive waste management.
- Helps identify bins likely to require collection soon.
- Supports proactive decision-making.
- Can improve resource planning.

Limitations:

- Requires sufficient historical data.
- Prediction accuracy depends on data quality.
- Unusual events can make predictions less accurate.
- Model development and maintenance require technical expertise.

This approach is highly relevant because the proposed system aims to move from simple monitoring toward predictive waste management.

4. Smart Waste Collection and Route Optimisation

Some intelligent waste-management approaches combine bin-status information with route optimisation. Collection vehicles can be directed toward bins that require immediate attention.

Advantages:

- Can reduce unnecessary vehicle movement.
- Improves collection efficiency.
- Supports priority-based collection.
- Can reduce fuel consumption and operational costs.

Limitations:

- Requires reliable and continuously updated bin information.
- Route optimisation can become complex when many bins and vehicles are involved.

- May require integration with GPS and fleet-management systems.

This approach can be considered as a future extension of the proposed system.

5. Commercial Smart Waste-Management Solutions

Commercial smart waste-management platforms use connected sensors, cloud platforms, dashboards, and data analytics to monitor waste containers and improve collection operations.

These solutions demonstrate that smart waste monitoring is technically feasible and can provide benefits such as improved visibility, reduced unnecessary collections, and better operational planning. However, commercial systems can involve significant deployment, subscription, hardware, and maintenance costs and may be designed primarily for large-scale organisations.

The proposed academic project focuses on developing a simplified and cost-effective prototype that demonstrates the core principles of real-time monitoring and AI-based prediction.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Conventional scheduled collection	Simple and easy to operate	Fixed schedules, unnecessary trips, overflowing bins	Provides the baseline approach for comparison
Manual monitoring	Low technology requirement	Time-consuming, inaccurate, no continuous monitoring	Shows the need for automation
IoT-based smart bins	Real-time fill-level monitoring and alerts	Mainly provides current status; sensor dependency	Provides real-time data for the proposed system
Sensor-based waste monitoring	Automatic measurement of waste level	Sensor errors and maintenance requirements	Used for collecting bin-status information
AI-based waste prediction	Predicts future waste accumulation	Requires historical and quality data	Forms the intelligence component of the

			proposed system
Route optimisation systems	Reduces travel and collection costs	Complex for multiple vehicles and locations	Can be integrated as a future enhancement
Commercial smart waste platforms	Advanced monitoring and analytics	Higher deployment and maintenance costs	Provides reference for practical system design
Proposed AI-enhanced system	Combines monitoring, prediction, alerts, and prioritisation	Prototype-scale implementation and data dependency	Provides an integrated and cost-effective solution

Table 1 : Comparative analysis

2.4 Research / Engineering Gap

The literature review indicates that several technologies are available for improving waste management, but important challenges remain.

Many conventional systems depend on fixed collection schedules, while basic IoT systems primarily provide information about the current fill level of a waste bin. Although AI-based prediction can estimate future waste accumulation, it often requires sufficient historical data and may not be integrated with real-time monitoring and collection prioritisation.

Therefore, there is an engineering gap in developing a simple, integrated, and cost-effective system that combines real-time waste monitoring with AI-based prediction and collection alerts.

The major unresolved issues include:

1. Lack of integration between real-time monitoring and prediction.
2. Limited prediction of when bins will actually require collection.
3. Inefficient prioritisation of bins requiring immediate attention.
4. Dependence on fixed collection schedules in conventional systems.
5. Data-quality and sensor-reliability challenges.
6. Cost and complexity of large-scale commercial solutions.

2.5 Proposed Contribution

The proposed AI-Enhanced Smart Waste Management System addresses the identified gap by integrating real-time waste-bin monitoring, data processing, AI-based prediction, and collection prioritisation into a single system.

The proposed system is designed to:

- Monitor the fill level of waste bins in real time.
- Collect and process historical waste accumulation data.
- Predict future waste accumulation using AI techniques.
- Identify bins approaching their capacity.
- Generate alerts when collection is required.
- Prioritise bins according to their urgency.
- Support condition-based rather than purely schedule-based collection.
- Provide a simple monitoring interface for decision-making.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Municipal Waste Management Authority	Real-time information about waste-bin status
Waste Collection Workers	Clear identification of bins requiring collection
System Administrator	Easy monitoring and management of system data
Public / Residents	Reduced overflowing bins and cleaner surroundings
Project Operator	Reliable, low-cost and easy-to-maintain prototype

Table 2 : Stakeholder / User Requirements for the AI-Enhanced Smart Waste Management System

Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Measure waste-bin fill level	Functional	Detect fill level from 0–100%
Monitor bin status	Functional	Update status at regular intervals
Predict waste accumulation	Functional	Provide prediction using historical data
Generate collection alerts	Functional	Alert when bin reaches $\geq 80\%$ capacity
Prioritise bins	Functional	Classify bins as Normal, Warning or Critical
Store monitoring data	Functional	Maintain timestamped bin records
System response time	Non-Functional	Alert generated within 5 seconds of receiving data
Prediction accuracy	Non-Functional	Target $\geq 85\%$ on test data
System availability	Non-Functional	$\geq 95\%$ during prototype testing
Ease of use	Non-Functional	Simple dashboard with clear status indicators
Scalability	Non-Functional	Support at least 20 monitored bins in prototype

Table 3 : Functional & Non-Functional Requirements

3.2 Constraints & Applicable Standards

Project Constraints

The following constraints are applicable to the project:

- **Economic:** The prototype should use low-cost sensors and computing components.
- **Technical:** Sensor accuracy and communication reliability may affect the system.

- **Environmental:** Components should operate under normal indoor/outdoor conditions considered during prototype testing.
- **Data:** AI prediction performance depends on the quantity and quality of historical waste data.
- **Safety:** Electrical and electronic components must be safely enclosed and protected from direct contact with waste.
- **Privacy:** The system should avoid collecting unnecessary personal information.

Applicable Standards / Codes

Standard / Code	Requirement	How Applied in This Project
ISO 14001:2015	Environmental management principles	The project supports improved waste-management efficiency and reduced unnecessary collection activities.
ISO 31000:2018	Risk management principles	Risks such as sensor failure, data loss and incorrect prediction are identified and mitigated.
IEC 60529 (IP Code)	Protection of electrical enclosures against solids and water	Relevant enclosure protection is considered when designing a practical smart-bin prototype.
ISO/IEC 27001:2022	Information security management	Security principles are considered for protecting stored monitoring data and system access.

Note: These standards are used as design references where applicable; the student prototype is not claimed to be formally certified against them.

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Bin fill-level measurement	0–100	%	High
Collection alert threshold	≥ 80	%	High
Critical threshold	≥ 90	%	High
Monitoring update interval	≤ 60	seconds	Medium
Alert response time	≤ 5	seconds	High
AI prediction accuracy	≥ 85	%	High
Number of prototype bins	≥ 20	bins	Medium
System availability during testing	≥ 95	%	High
Dashboard accessibility	Real-time status	—	High
Data storage	Timestamped records	—	Medium

Table 4 : Design Specifications

3.4 Alternative Solutions & Evaluation

Alternative 1: Conventional Fixed-Schedule Collection

Waste bins are collected according to predetermined schedules without continuous monitoring. This approach is simple and inexpensive but cannot respond effectively to variations in waste generation.

Alternative 2: IoT-Based Smart Bin Monitoring

Sensors continuously measure the fill level of waste bins and send the information to a monitoring platform. It provides real-time information but has limited capability for predicting future waste requirements.

Alternative 3: AI-Enhanced Smart Waste Management

The proposed approach combines real-time IoT monitoring with AI-based waste prediction and collection prioritisation. It can identify current bin conditions and estimate which bins are likely to require collection soon.

Evaluation Matrix

Scoring: 1 = Poor, 3 = Moderate, 5 = Excellent

Criterion	Weight	Alternative 1: Fixed Schedule	Alternative 2: IoT Monitoring	Alternative 3: AI-Enhanced
Performance	30%	2	4	5
Cost	20%	5	4	3
Safety	15%	3	4	4
Sustainability	20%	2	4	5
Reliability	15%	4	4	4
Weighted Score	100%	2.95	4.00	4.35

The **AI-Enhanced approach** achieves the highest weighted score because it provides better predictive capability and sustainability while maintaining acceptable cost, safety and reliability.

3.5 Selected Design & Detailed Design

Final Design Selection

Alternative 3, the AI-Enhanced Smart Waste Management System, is selected because it provides the highest overall weighted score. Unlike fixed-schedule collection, it uses actual bin conditions, while unlike basic IoT monitoring, it can use historical data to predict future waste accumulation.

System Architecture

The system consists of the following major subsystems:

Smart Bin → Sensors → Data Collection → Data Processing → AI Prediction → Decision Module → Alert / Dashboard

You can place this architecture diagram in this section:

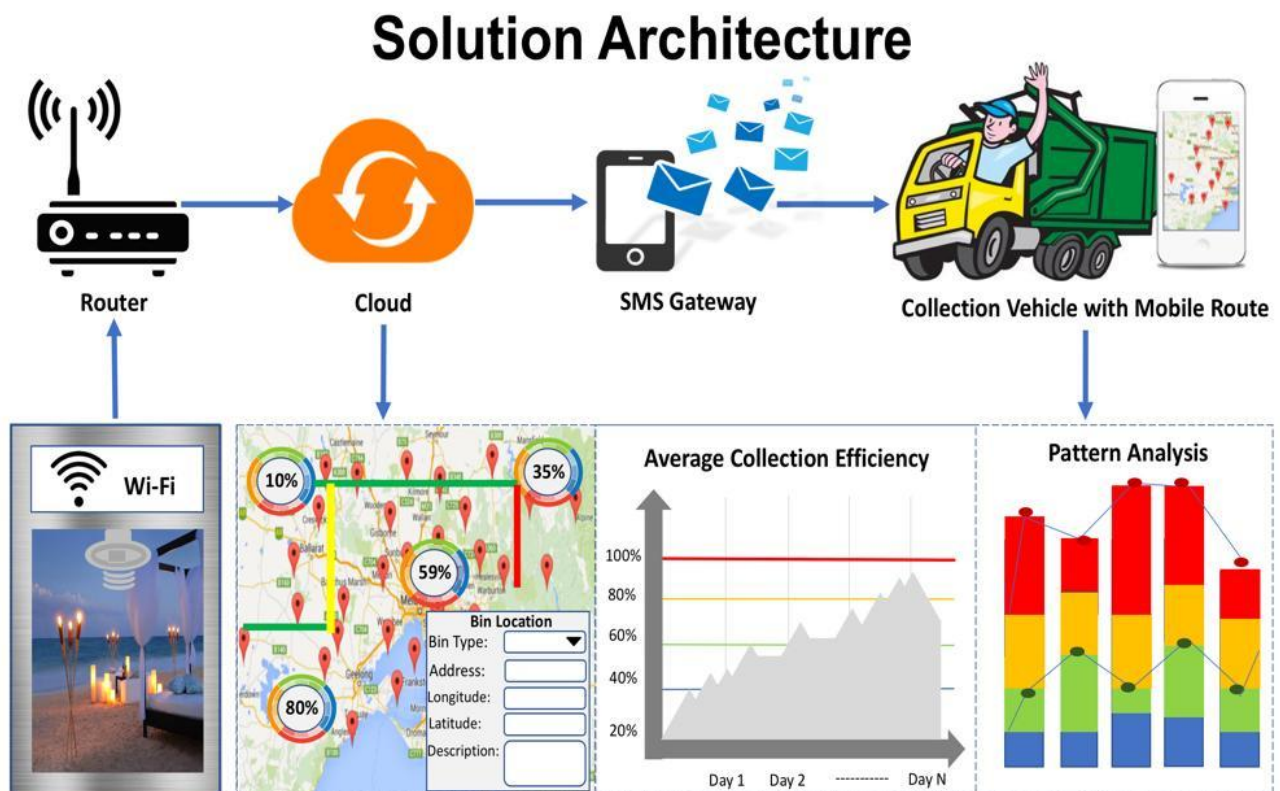
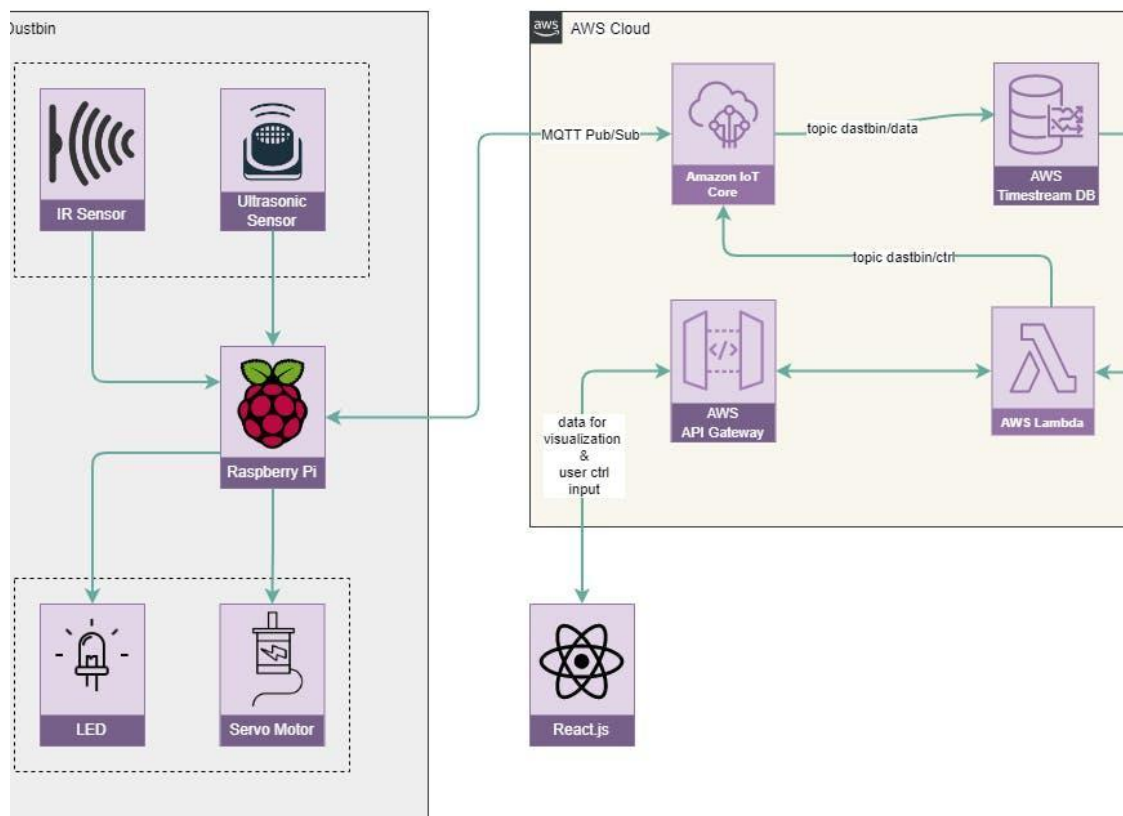
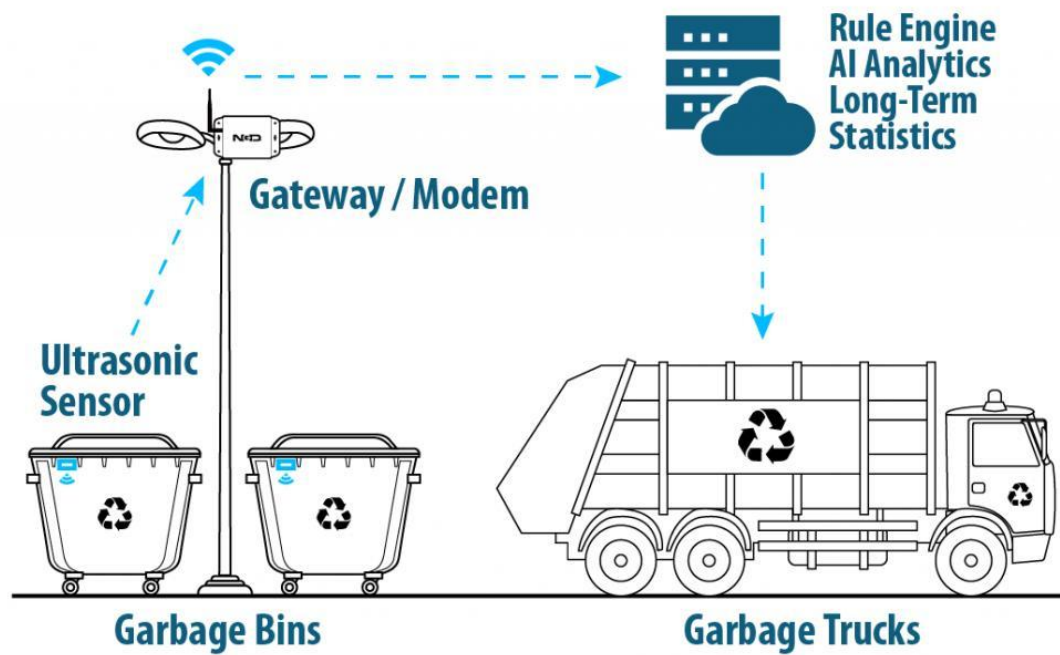


Figure 1: Smart Waste Management System Architecture



```

from flask import Flask, render_template, request, jsonify, Response
import mysql.connector
import os, csv, io, math, random
from datetime import datetime, timedelta

app = Flask(__name__)
app.secret_key = os.getenv('SECRET_KEY', 'smart-waste-demo')
DB = {
    'host': os.getenv('DB_HOST', 'localhost'),
    'user': os.getenv('DB_USER', 'root'),
    'password': os.getenv('DB_PASSWORD', ''),
    'database': os.getenv('DB_NAME', 'smart_waste'),
}

def server_db():
    return mysql.connector.connect(host=DB['host'], user=DB['user'], password=DB['password'])

def db():
    return mysql.connector.connect(**DB)

def columns(cur, table):
    cur.execute(f"SHOW COLUMNS FROM `{table}`")
    return {row[0] for row in cur.fetchall()}

def add_column(cur, table, name, definition):
    if name not in columns(cur, table):
        cur.execute(f"ALTER TABLE `{table}` ADD COLUMN `{name}` {definition}")

def ensure_schema():
    c = server_db(); q = c.cursor()
    q.execute(f"CREATE DATABASE IF NOT EXISTS `{DB['database']}`")
    q.close(); c.close()
    c = db(); q = c.cursor()
    q.execute("""CREATE TABLE IF NOT EXISTS users (
        id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT NULL
        DEFAULT 'User',
        email VARCHAR(255) NOT NULL UNIQUE, password VARCHAR(255) NOT NULL, role
        VARCHAR(50) DEFAULT 'operator'""")
    q.execute("""CREATE TABLE IF NOT EXISTS bins (
        id INT AUTO_INCREMENT PRIMARY KEY, bin_code VARCHAR(50) NOT NULL
        UNIQUE,
        location_name VARCHAR(150) NOT NULL, latitude DECIMAL(10,6) DEFAULT
        13.082700,
        longitude DECIMAL(10,6) DEFAULT 80.270700, capacity_percent DECIMAL(5,2)
    """)

```

```

DEFAULT 0,
    battery_percent DECIMAL(5,2) DEFAULT 100, temperature DECIMAL(6,2) DEFAULT 28,
    gas_level DECIMAL(8,2) DEFAULT 10, waste_type VARCHAR(80) DEFAULT 'Mixed',
    status VARCHAR(30) DEFAULT 'ACTIVE', last_seen DATETIME NULL)""")
q.execute("""CREATE TABLE IF NOT EXISTS alerts (
    id INT AUTO_INCREMENT PRIMARY KEY, bin_id INT NULL, alert_type
VARCHAR(80) DEFAULT 'System',
    severity VARCHAR(30) DEFAULT 'INFO', message VARCHAR(255) DEFAULT "",
    is_resolved TINYINT(1) DEFAULT 0, created_at DATETIME DEFAULT
CURRENT_TIMESTAMP)""")
q.execute("""CREATE TABLE IF NOT EXISTS sensor_readings (
    id INT AUTO_INCREMENT PRIMARY KEY, bin_id INT, fill_level DECIMAL(5,2)
DEFAULT 0,
    battery_percent DECIMAL(5,2) DEFAULT 100, temperature DECIMAL(6,2) DEFAULT 28,
    gas_level DECIMAL(8,2) DEFAULT 10, created_at DATETIME DEFAULT
CURRENT_TIMESTAMP)""")
q.execute("""CREATE TABLE IF NOT EXISTS collection_schedules (
    id INT AUTO_INCREMENT PRIMARY KEY, bin_id INT NOT NULL, scheduled_for
DATETIME NOT NULL,
    vehicle VARCHAR(80) DEFAULT 'Truck-01', driver VARCHAR(100) DEFAULT
'Assigned',
    status VARCHAR(30) DEFAULT 'PLANNED', notes VARCHAR(255) DEFAULT """)""")
q.execute("""CREATE TABLE IF NOT EXISTS system_settings (
    setting_key VARCHAR(100) PRIMARY KEY, setting_value VARCHAR(255) NOT
NULL)""")
for n,d in {'bin_id':'INT NOT NULL','scheduled_for':'DATETIME
NULL','vehicle':"VARCHAR(80) DEFAULT 'Truck-01'",'driver':"VARCHAR(100) DEFAULT
'Assigned'",'status':"VARCHAR(30) DEFAULT 'PLANNED'",'notes':"VARCHAR(255)
DEFAULT ""'}.items():
    add_column(q,'collection_schedules',n,d)
# Migrate common legacy schemas.
for n,d in {
    'name':"VARCHAR(100) NOT NULL DEFAULT 'User'", 'email':"VARCHAR(255) NULL",
    'password':"VARCHAR(255) NULL",'role':"VARCHAR(50) DEFAULT 'operator'"}.items():
add_column(q,'users',n,d)
for n,d in {
    'bin_code':"VARCHAR(50) NULL",'location_name':"VARCHAR(150) DEFAULT
'Unknown',
    'latitude':"DECIMAL(10,6) DEFAULT 13.082700",'longitude':"DECIMAL(10,6) DEFAULT
80.270700',
    'capacity_percent':"DECIMAL(5,2) DEFAULT 0','battery_percent':"DECIMAL(5,2)
DEFAULT 100',
    'temperature':"DECIMAL(6,2) DEFAULT 28','gas_level':"DECIMAL(8,2) DEFAULT 10',
    'waste_type':"VARCHAR(80) DEFAULT 'Mixed'",'status':"VARCHAR(30) DEFAULT
'ACTIVE'",'last_seen':"DATETIME NULL'}.items(): add_column(q,'bins',n,d)
for n,d in {'bin_id':'INT NULL','alert_type':"VARCHAR(80) DEFAULT
'System'",'severity':"VARCHAR(30) DEFAULT 'INFO'",'message':"VARCHAR(255) DEFAULT
'",'is_resolved':"TINYINT(1) DEFAULT 0','created_at':"DATETIME DEFAULT
CURRENT_TIMESTAMP'}.items():

```

```

        add_column(q,'alerts',n,d)
    for n,d in {'bin_id':'INT NULL','fill_level':'DECIMAL(5,2) DEFAULT
0','battery_percent':'DECIMAL(5,2) DEFAULT 100','temperature':'DECIMAL(6,2) DEFAULT
28','gas_level':'DECIMAL(8,2) DEFAULT 10','created_at':'DATETIME DEFAULT
CURRENT_TIMESTAMP'}.items(): add_column(q,'sensor_readings',n,d)
    bcols=columns(q,'bins')
    if 'location' in bcols:
        try:
            q.execute("ALTER TABLE bins MODIFY COLUMN location VARCHAR(150) NULL
DEFAULT 'Unknown'")
        except Exception:
            pass
        q.execute("UPDATE bins SET
location=COALESCE(NULLIF(location,''),location_name,'Unknown') WHERE location IS NULL
OR location=''")
        if 'fill_level' in bcols: q.execute("UPDATE bins SET
capacity_percent=COALESCE(fill_level,capacity_percent) WHERE capacity_percent IS NULL
OR capacity_percent=0")
        if 'fill_percentage' in bcols: q.execute("UPDATE bins SET
capacity_percent=COALESCE(fill_percentage,capacity_percent) WHERE capacity_percent IS
NULL OR capacity_percent=0")
        if 'location' in bcols: q.execute("UPDATE bins SET
location_name=COALESCE(NULLIF(location,''),location_name)")
        q.execute("UPDATE bins SET
location_name=COALESCE(location_name,'Unknown'),latitude=COALESCE(latitude,13.0827),lo
ngitude=COALESCE(longitude,80.2707),capacity_percent=COALESCE(capacity_percent,0),batter
y_percent=COALESCE(battery_percent,100),temperature=COALESCE(temperature,28),gas_level
=COALESCE(gas_level,10),waste_type=COALESCE(waste_type,'Mixed'),status=COALESCE(sta
tus,'ACTIVE')")
        q.execute("SELECT id FROM users WHERE LOWER(TRIM(email))='admin@example.com'
LIMIT 1")
        u=q.fetchone()
        if u: q.execute("UPDATE users SET name='Administrator',password='admin123',role='admin'
WHERE id=%s",(u[0],))
        else: q.execute("INSERT INTO users(name,email,password,role)
VALUES('Administrator','admin@example.com','admin123','admin')")
        q.execute('SELECT COUNT(*) FROM bins'); n=q.fetchone()[0]
        if n==0:
            sample=[('BIN-001','Anna Nagar',13.0878,80.2089,82,78,29,22,'Mixed','ACTIVE'),('BIN-
002','T Nagar',13.0418,80.2341,94,61,31,48,'Plastic','FULL'),('BIN-
003','Adyar',13.0067,80.2570,56,92,28,16,'Organic','ACTIVE'),('BIN-
004','Velachery',12.9815,80.2180,73,34,30,35,'Mixed','ACTIVE'),('BIN-
005','Guindy',13.0068,80.2206,28,88,27,8,'Paper','ACTIVE')]
            q.executemany("INSERT INTO
bins(bin_code,location_name,latitude,longitude,capacity_percent,battery_percent,temperature,gas_l
evel,waste_type,status,last_seen) VALUES(%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,NOW())",sample)
            # Seed realistic historical readings so prediction is immediately functional.
            q.execute('SELECT COUNT(*) FROM sensor_readings')
            if q.fetchone()[0] < 10:

```

```

        q.execute('SELECT id,capacity_percent,battery_percent,temperature,gas_level FROM bins
ORDER BY id')
        for bid,fill,bat,temp,gas in q.fetchall():
            for h in [24,18,12,6,3,1]:
                f=max(5,min(98,float(fill)-random.uniform(0.8,2.8)*h/6))
                q.execute(f"INSERT INTO
sensor_readings(bin_id,fill_level,battery_percent,temperature,gas_level,created_at)
VALUES(%s,%s,%s,%s,%s,NOW()-INTERVAL {int(h)} HOUR)",(bid,f,max(10,float(bat)-
random.uniform(0,5)),float(temp)+random.uniform(-1,1),max(0,float(gas)+random.uniform(-3,3))))
            defaults =
{'overflow_high':'80','overflow_critical':'90','battery_low':'25','gas_high':'40','route_threshold':'60','si
mulation_step_min':'1','auto_alerts':'1','collection_window_hours':'24','default_vehicle':'Truck-01'}
            for k,v in defaults.items():
                q.execute('INSERT IGNORE INTO system_settings(setting_key,setting_value)
VALUES(%s,%s)',(k,v))
            c.commit(); q.close(); c.close()
            generate_all_alerts()

def query(sql, params=()):
    c=db(); cur=c.cursor(dictionary=True)
    try: cur.execute(sql,params); return cur.fetchall()
    finally: cur.close(); c.close()

def exec_sql(sql, params=(), many=False):
    c=db(); cur=c.cursor()
    try:
        if many: cur.executemany(sql,params)
        else: cur.execute(sql,params)
        c.commit(); return cur.lastrowid
    finally: cur.close(); c.close()

def clamp(v,a,b): return max(a,min(b,float(v)))

def prediction_for(binrow):
    bid=binrow['id']; fill=float(binrow['capacity_percent'] or 0)
    hist=query('SELECT fill_level,created_at FROM sensor_readings WHERE bin_id=%s ORDER
BY created_at DESC LIMIT 12',(bid,))
    points=[]
    now=datetime.now()
    for r in hist:
        dt=r['created_at']; hours=(now-dt).total_seconds()/3600 if dt else 0
        points.append((hours,float(r['fill_level'] or 0)))
    # Regression of fill against elapsed time. Positive slope = filling rate.
    slope=0.0
    if len(points)>=2:
        xs=[p[0] for p in points]; ys=[p[1] for p in points]; xm=sum(xs)/len(xs); ym=sum(ys)/len(ys)

```

```

    den=sum((x-xm)**2 for x in xs)
    slope=sum((x-xm)*(y-ym) for x,y in zip(xs,ys))/den if den else 0
    slope=-slope # elapsed hours runs backwards from newest reading
    slope=max(0.0,slope)
    if slope<0.15: slope=max(0.35,fill/160)
    hours=max(0.1,(100-fill)/slope) if fill<100 else 0.1
    # Probability of overflow within 24h using a smooth risk curve.
    risk=1/(1+math.exp(-(fill-72)/9))
    if hours<24: risk=max(risk,0.78)
    if hours<8: risk=max(risk,0.93)
    confidence=min(0.95,0.45+0.05*len(points))
    if fill>=90: risk=0.99
    if fill>=80: level='CRITICAL' if hours<8 else 'HIGH'
    elif fill>=60: level='MEDIUM'
    else: level='LOW'
    return
    {'bin_id':bid,'bin_code':binrow['bin_code'],'location_name':binrow['location_name'],'current_fill':round(fill,1),'rate_per_hour':round(slope,2),'hours_to_full':round(hours,1),'probability_24h':round(risk,3),'confidence':round(confidence,2),'risk_level':level,'recommendation':('Collect within 8 hours' if hours<8 else 'Schedule collection within 24 hours' if hours<24 else 'Continue monitoring'),'samples':len(points)}

```

```

def get_setting(key, default=None):
    """Read a system setting safely; return default if unavailable."""
    try:
        rows = query('SELECT setting_value FROM system_settings WHERE setting_key=%s LIMIT 1',(key,))
        return rows[0]['setting_value'] if rows else default
    except Exception:
        return default

```

```

def refresh_alerts(bin_id, fill, battery, gas):
    high=float(get_setting('overflow_high','80')); critical=float(get_setting('overflow_critical','90'));
    lowbat=float(get_setting('battery_low','25')); gas_high=float(get_setting('gas_high','40'))
    rules=[]
    if fill>=critical: rules.append(('Overflow','CRITICAL',f'Bin is {fill:.0f}% full — immediate collection required'))
    elif fill>=high: rules.append(('Overflow','HIGH',f'Bin is {fill:.0f}% full — collection recommended'))
    if battery<=lowbat: rules.append(('Battery','HIGH',f'Device battery is {battery:.0f}%'))
    if gas>=gas_high: rules.append(('Gas','HIGH',f'Gas level is {gas:.0f} — inspection recommended'))
    for typ,sev,msg in rules:
        if not query('SELECT id FROM alerts WHERE bin_id=%s AND alert_type=%s AND is_resolved=0 LIMIT 1',(bin_id,typ)):
            exec_sql('INSERT INTO alerts(bin_id,alert_type,severity,message) VALUES(%s,%s,%s,%s)',(bin_id,typ,sev,msg))

```

```

def generate_all_alerts():
    """Re-evaluate every bin and create any missing active alerts."""
    rows = query("""SELECT id, capacity_percent, battery_percent, gas_level FROM bins""")
    for row in rows:
        try:
            refresh_alerts(
                row['id'],
                float(row.get('capacity_percent') or 0),
                float(row.get('battery_percent') or 100),
                float(row.get('gas_level') or 0),
            )
        except (TypeError, ValueError, KeyError):
            continue

@app.errorhandler(Exception)
def handle_unexpected_error(e):
    if request.path.startswith('/api/'):
        return jsonify(ok=False, message=str(e), error=type(e).__name__), 500
    return str(e), 500

@app.route('/')
def home(): return render_template('index.html')

@app.get('/api/health')
def health():
    try: ensure_schema(); return jsonify(ok=True,database=DB['database'])
    except Exception as e: return jsonify(ok=False,message=str(e)),500

@app.post('/api/login')
def login():
    d=request.get_json(silent=True) or {}; email=str(d.get('email','')).strip().lower();
    password=str(d.get('password',''))
    try:
        ensure_schema(); rows=query('SELECT id,name,email,role,password FROM users WHERE
LOWER(TRIM(email))=%s LIMIT 1',(email,)); u=rows[0] if rows else None
        if u and str(u['password'])==password: u.pop('password',None); return jsonify(ok=True,user=u)
        return jsonify(ok=False,message='Invalid email or password'),401
    except Exception as e:return jsonify(ok=False,message='Database error: '+str(e)),500

@app.get('/api/bins')
def get_bins():
    rows=query('SELECT * FROM bins ORDER BY id')
    for x in rows:
        for k in ['latitude','longitude','capacity_percent','battery_percent','temperature','gas_level']:
            x[k]=float(x.get(k) or 0)

```



```

sensor_readings WHERE bin_id=%s',(bin_id,)); exec_sql('DELETE FROM collection_schedules
WHERE bin_id=%s',(bin_id,)); exec_sql('DELETE FROM bins WHERE id=%s',(bin_id,)); return
jsonify(ok=True)
except Exception as e: return jsonify(ok=False,message=str(e)),400

@app.get('/api/dashboard')
def dashboard():
    b=query('SELECT * FROM bins ORDER BY id')
    for x in b:
        for k in ['capacity_percent','battery_percent','temperature','gas_level']: x[k]=float(x.get(k) or 0)
        full=sum(x['capacity_percent']>=90 for x in b); off=sum(str(x['status']).upper()=='OFFLINE' for
x in b); alerts=int(query('SELECT COUNT(*) n FROM alerts WHERE is_resolved=0')[0]['n'])
        preds=[prediction_for(x) for x in b]
        return jsonify(total=len(b),full=full,offline=off,alerts=alerts,bins=b,predictions=preds)

@app.get('/api/predictions')
def predictions(): return jsonify([prediction_for(x) for x in query('SELECT * FROM bins ORDER
BY id')])

@app.get('/api/recommendations')
def recommendations():
    return
    jsonify([{'bin_code':p['bin_code'],'priority':p['risk_level'],'recommendation':p['recommendation']}
for p in [prediction_for(x) for x in query('SELECT * FROM bins ORDER BY id')]])

@app.get('/api/alerts')
def alerts(): return jsonify(query("""SELECT a.id,a.bin_id,COALESCE(b.bin_code,'N/A')
bin_code,a.alert_type,a.severity,a.message,a.is_resolved,a.created_at FROM alerts a LEFT JOIN
bins b ON a.bin_id=b.id ORDER BY a.is_resolved,a.created_at DESC LIMIT 100"""))

@app.post('/api/alerts/<int:alert_id>/resolve')
def resolve_alert(alert_id):
    exec_sql('UPDATE alerts SET is_resolved=1 WHERE id=%s',(alert_id,)); return
    jsonify(ok=True)

@app.get('/api/iot')
def get_iot(): return jsonify(query("""SELECT
s.id,s.bin_id,b.bin_code,s.fill_level,s.battery_percent,s.temperature,s.gas_level,s.created_at FROM
sensor_readings s LEFT JOIN bins b ON s.bin_id=b.id ORDER BY s.created_at DESC LIMIT
100"""))

@app.post('/api/iot')
def iot():
    d=request.get_json(silent=True) or {}
    try:
        bid=int(d['bin_id']); fill=clamp(d['fill_level'],0,100);
        battery=clamp(d.get('battery_percent',100),0,100); temp=float(d.get('temperature',28));
        gas=max(0,float(d.get('gas_level',10)))
        exec_sql('INSERT INTO

```

```

sensor_readings(bin_id,fill_level,battery_percent,temperature,gas_level)
VALUES(%s,%s,%s,%s,%s)',(bid,fill,battery,temp,gas))
    exec_sql("UPDATE bins SET
capacity_percent=%s,battery_percent=%s,temperature=%s,gas_level=%s,last_seen=NOW(),status=
%s WHERE id=%s",(fill,battery,temp,gas,'FULL' if fill>=90 else 'ACTIVE',bid));
refresh_alerts(bid,fill,battery,gas)
    return jsonify(ok=True,prediction=prediction_for(query('SELECT * FROM bins WHERE
id=%s',(bid,))[0]))
    except Exception as e:return jsonify(ok=False,message=str(e)),400

@app.post('/api/iot/simulate')
def simulate_iot():
    bins=query('SELECT * FROM bins ORDER BY id'); updated=[]
    for b in bins:
        fill=clamp(float(b['capacity_percent'])+random.uniform(1.0,7.0),0,100)
        battery=clamp(float(b['battery_percent'])-random.uniform(0.3,2.0),5,100)
        temp=float(b['temperature'])+random.uniform(-0.6,0.9);
    gas=max(0,float(b['gas_level'])+random.uniform(-3,5))
    exec_sql('INSERT INTO
sensor_readings(bin_id,fill_level,battery_percent,temperature,gas_level)
VALUES(%s,%s,%s,%s,%s)',(b['id'],fill,battery,temp,gas))
    exec_sql("UPDATE bins SET
capacity_percent=%s,battery_percent=%s,temperature=%s,gas_level=%s,last_seen=NOW(),status=
%s WHERE id=%s",(fill,battery,temp,gas,'FULL' if fill>=90 else 'ACTIVE',b['id']));
refresh_alerts(b['id'],fill,battery,gas); updated.append({'bin_code':b['bin_code'],'fill':round(fill,1)})
    return jsonify(ok=True,updated=updated)

@app.get('/api/route')
def route():
    threshold=float(get_setting('route_threshold','60'))
    bins=query('SELECT id,bin_code,location_name,latitude,longitude,capacity_percent FROM bins
WHERE capacity_percent >= %s ORDER BY capacity_percent DESC',(threshold,))
    if not bins:return jsonify(strategy=f'No collection required above
{threshold:.0f}%',{stops:[],total_km=0})
    current=(13.0827,80.2707); remaining=bins[:]; stops=[]; total=0
    def dist(a,b):
        R=6371; p1,p2=math.radians(float(a[0])),math.radians(float(b[0]));
        dp=math.radians(float(b[0])-float(a[0])); dl=math.radians(float(b[1])-float(a[1]));
        h=math.sin(dp/2)**2+math.cos(p1)*math.cos(p2)*math.sin(dl/2)**2; return
        2*R*math.asin(math.sqrt(h))
    while remaining:
        idx=max(range(len(remaining)),key=lambda i:float(remaining[i]['capacity_percent'])) if not
stops else min(range(len(remaining)),key=lambda
i:dist(current,remaining[i]['latitude'],remaining[i]['longitude']))
        b=remaining.pop(idx); leg=dist(current,(b['latitude'],b['longitude'])); total+=leg;
current=(float(b['latitude']),float(b['longitude']))

stops.append({'bin_code':b['bin_code'],'location_name':b['location_name'],'capacity_percent':float(b
['capacity_percent'],'latitude':float(b['latitude'],'longitude':float(b['longitude'],'leg_km':round(leg,2

```

```

}))
    return jsonify(strategy=f'Greedy nearest-neighbour with highest-fill priority (threshold
{threshold:.0f}%)',stops=stops,total_km=round(total,2))

@app.get('/api/scheduling')
def scheduling():
    rows=query("""SELECT
s.id,b.bin_code,b.location_name,b.capacity_percent,s.scheduled_for,s.vehicle,s.driver,s.status,s.note
s FROM collection_schedules s JOIN bins b ON b.id=s.bin_id ORDER BY s.scheduled_for""")
    for r in rows:
        r['capacity_percent']=float(r.get('capacity_percent') or 0)
        if isinstance(r.get('scheduled_for'), datetime):
            r['scheduled_for']=r['scheduled_for'].strftime('%Y-%m-%d %H:%M')
    return jsonify(rows)

@app.post('/api/scheduling')
def add_schedule():
    d=request.get_json(silent=True) or {}
    try:
        bid=int(d.get('bin_id')); when=str(d.get('scheduled_for',)).strip()
        if not when:return jsonify(ok=False,message='Please select a date and time'),400
        datetime.strptime(when,'%Y-%m-%d %H:%M')
        sid=exec_sql('INSERT INTO
collection_schedules(bin_id,scheduled_for,vehicle,driver,status,notes)
VALUES(%s,%s,%s,%s,%s,%s)',(bid,when,d.get('vehicle') or get_setting('default_vehicle','Truck-
01'),d.get('driver') or 'Assigned','PLANNED',d.get('notes','')))
        return jsonify(ok=True,id=sid)
    except Exception as e:return jsonify(ok=False,message=str(e)),400

@app.get('/api/users')
def users(): return jsonify(query('SELECT id,name,email,role FROM users ORDER BY id'))

@app.post('/api/users')
def add_user():
    d=request.get_json(silent=True) or {}; exec_sql('INSERT INTO
users(name,email,password,role)
VALUES(%s,%s,%s,%s)',(d['name'],d['email'].strip().lower(),d['password'],d.get('role','operator')));
    return jsonify(ok=True)

@app.get('/api/analytics')
def analytics():
    daily=query("""SELECT DATE(created_at) day,ROUND(AVG(fill_level),1)
avg_fill,ROUND(MAX(fill_level),1) peak_fill,COUNT(*) readings FROM sensor_readings
WHERE created_at>=NOW()-INTERVAL 7 DAY GROUP BY DATE(created_at) ORDER BY
day""")
    return jsonify(daily=daily)

@app.get('/api/settings')
def settings():

```

```

    rows=query('SELECT setting_key,setting_value FROM system_settings ORDER BY
setting_key')
    return jsonify({'system':{'database':DB['database'],'host':DB['host'],'user':DB['user'],'mode':'Local
MySQL','status':'Connected'},'thresholds':{r['setting_key']:r['setting_value'] for r in rows}})

@app.post('/api/settings')
def save_settings():
    d=request.get_json(silent=True) or {};
    allowed={'overflow_high','overflow_critical','battery_low','gas_high','route_threshold','simulation_s
tep_min','auto_alerts','collection_window_hours','default_vehicle'}
    try:
        for k,v in d.items():
            if k in allowed:exec_sql('INSERT INTO system_settings(setting_key,setting_value)
VALUES(%s,%s) ON DUPLICATE KEY UPDATE
setting_value=VALUES(setting_value)',(k,str(v)))
            generate_all_alerts(); return jsonify(ok=True)
    except Exception as e:return jsonify(ok=False,message=str(e)),400

@app.post('/api/alerts/generate')
def generate_alerts_endpoint():
    generate_all_alerts(); return jsonify(ok=True)

@app.get('/api/reports')
def reports():
    d=dashboard().get_json(); return jsonify(generated_at=datetime.now().strftime('%Y-%m-%d
%H:%M:%S'),summary=d)

@app.get('/api/reports.csv')
def reports_csv():
    rows=query('SELECT
bin_code,location_name,capacity_percent,battery_percent,temperature,gas_level,waste_type,status,l
ast_seen FROM bins ORDER BY id'); s=io.StringIO(); fields=list(rows[0].keys()) if rows else
['bin_code','location_name','capacity_percent','battery_percent','temperature','gas_level','waste_type',
'status','last_seen']; w=csv.DictWriter(s,fieldnames=fields); w.writeheader(); w.writerows(rows);
return Response(s.getvalue(),mimetype='text/csv',headers={'Content-Disposition':'attachment;
filename=smartwaste_report.csv'})

if __name__=='__main__':
    try: ensure_schema(); print("\n🔄 Smart Waste Management System\nOpen:
http://127.0.0.1:5000\n")
    except Exception as e: print("\n⚠ Database setup failed:',e,'\nSet DB_PASSWORD if
required.\n")
    app.run(host='127.0.0.1',port=5000,debug=False)

```

Systems Approach

The sensing subsystem measures the waste level inside each bin and sends the collected information to the data-processing subsystem. The processing subsystem cleans and analyses the incoming data before passing relevant information to the AI prediction module. The AI module estimates future waste accumulation and provides its output to the decision module. The decision module determines the priority of each bin and generates alerts through the monitoring dashboard.

Key Engineering Calculation

For a bin with total usable height H and measured empty space d :

$$\text{Fill Level}(\%) = \frac{H-d}{H} \times 100$$

For example, if the bin height is **100 cm** and the measured empty space is **20 cm**:

$$\text{Fill Level} = \frac{100-20}{100} \times 100 = 80\%$$

Therefore, the system generates a **collection warning** when the fill level reaches the predefined 80% threshold.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Collection method	Fixed schedule	Simple and low cost	May collect half-empty bins	Use condition-based collection because it improves efficiency
Monitoring	Manual inspection	No sensor cost	Time-consuming and inconsistent	Use automated monitoring
Sensor-based measurement	Ultrasonic sensor	Non-contact measurement	Can be affected by irregular waste surfaces	Selected for prototype due to simplicity
Intelligence	Threshold only	Easy to implement	Cannot predict future conditions	Add AI prediction for proactive collection
Data display	Basic alerts	Simple implementation	Limited information	Use dashboard for easier monitoring
AI model	Complex deep learning	Potentially high accuracy	Requires large dataset and computing resources	Use a lightweight ML model suitable for available data

3.7 Sustainability, Ethics, Safety, Risk & Security

Sustainability, Ethics, Safety and Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Reduce unnecessary collection trips	Comparison of fixed schedules with condition-based collection	Selected condition-based monitoring to reduce unnecessary vehicle operation
Sustainability	Reduce overflowing bins	Fill-level monitoring and prediction	80% warning threshold introduced to support timely collection
Ethics	Responsible use of data	Type of information collected by sensors	System collects bin-related data rather than unnecessary personal information
Ethics	Transparent decision-making	AI prediction and alert criteria	Thresholds and prediction outputs are made visible to system operators
Health & Safety	Prevent overflowing waste	Real-time fill-level monitoring	Critical alerts are generated for highly filled bins
Health & Safety	Electrical safety	Sensor and electronic component requirements	Components are enclosed and isolated from direct waste contact
Security	Protect system data	Access and data-storage requirements	Authentication and controlled access should be used
Security	Prevent unauthorised system access	Basic cybersecurity considerations	Access to the monitoring dashboard is restricted to authorised users

Table 5: Sustainability, Ethics, Safety and Security

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Sensor gives incorrect reading	Medium	High	High	Sensor calibration and periodic testing	Medium
Sensor failure	Medium	High	High	Failure detection and maintenance	Medium
Communication failure	Medium	Medium	Medium	Local data buffering and reconnection mechanism	Low
Incorrect AI prediction	Medium	High	High	Model validation and threshold-based backup	Medium
Insufficient training data	Medium	Medium	Medium	Data collection and preprocessing	Low
Overflow due to missed alert	Low	High	Medium	Multiple threshold levels and alerts	Low
Power failure	Medium	Medium	Medium	Backup power or safe shutdown mechanism	Low
Unauthorised	Low	High	Medium	Authentication and	Low

dashboard access				access control	
Electronic components exposed to waste/moisture	Medium	High	High	Protective enclosure and suitable placement	Low
Data loss	Low	Medium	Low	Regular data backup	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The proposed AI-enhanced waste management system was developed using a modular approach consisting of waste-level monitoring, data collection, prediction, alert generation, and visualization modules. The system uses sensor-based data to monitor garbage-bin fill levels and applies data analysis and machine learning techniques to predict waste accumulation and support timely collection. The software modules were implemented and tested individually before integrating them into the complete system. The development process followed an iterative approach in which the system was tested, evaluated, and improved based on the obtained results.

Resource	Purpose
Garbage-bin fill-level dataset	Training and testing the prediction system
Ultrasonic/fill-level sensor data	Monitoring the amount of waste in the bin
Python	System implementation and data processing
Machine Learning algorithms	Waste generation prediction
Data visualization tools	Presenting monitoring and prediction results
Computer/Laptop	Development and testing
IoT components, where applicable	Collecting real-time bin information

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Software Implementation

The software implementation consists of data preprocessing, waste-level analysis, prediction, threshold-based alert generation, and result visualization. Python was used as the primary programming language because it provides libraries for data processing, machine learning, and visualization. The collected or simulated waste-level data was processed and used to identify the garbage-bin status and predict the requirement for waste collection.

Hardware Implementation

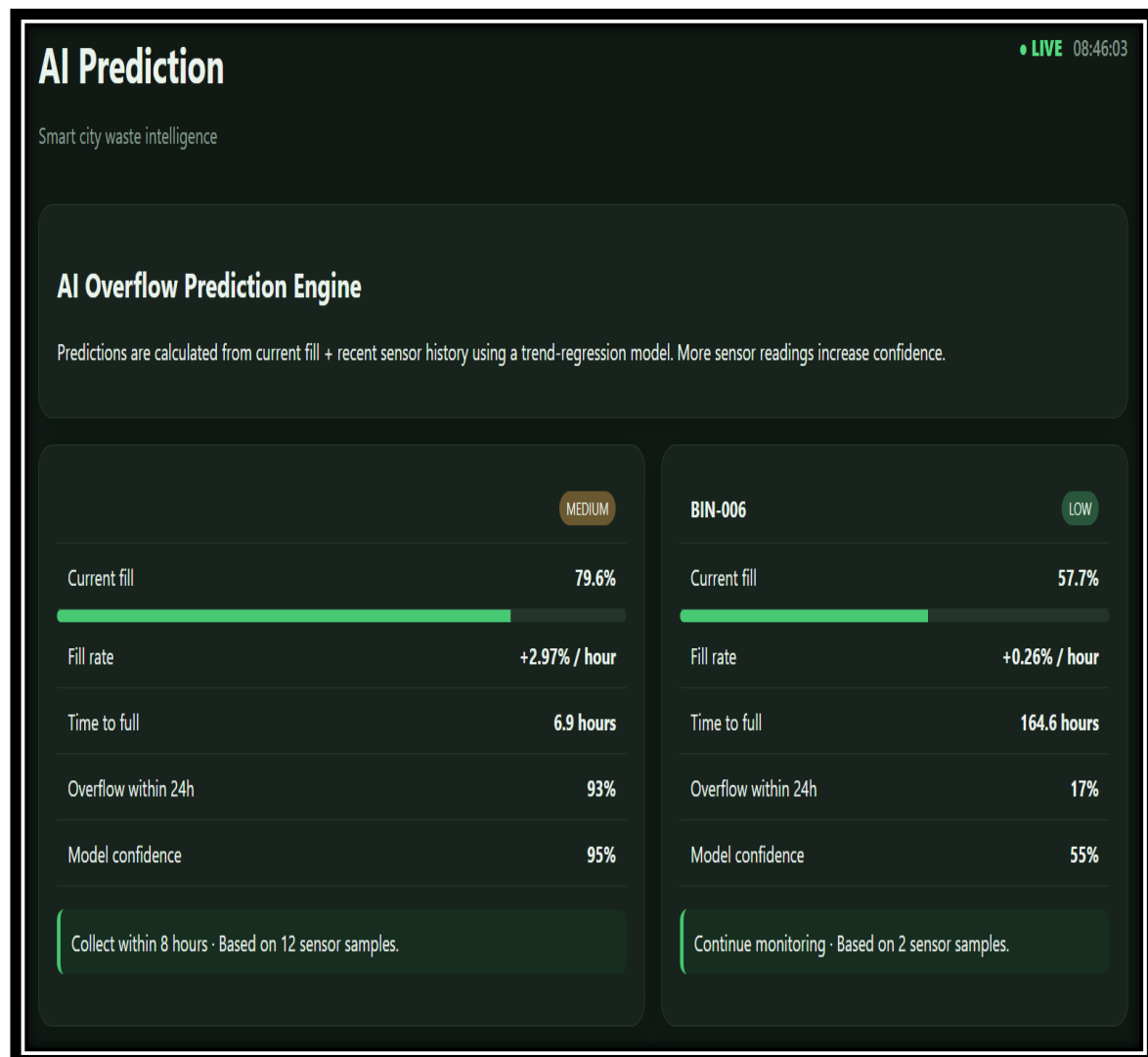
Where applicable, an ultrasonic or fill-level sensor can be connected to the garbage bin to measure the distance between the sensor and the waste surface. The measured value is converted into a percentage fill level. When the fill level reaches the predefined threshold, the system generates a collection alert.

Modern Tools Used

Tool / Software / Equipment	Purpose	Stage Used
Python	Programming and system implementation	Development
Google Colab / Jupyter Notebook	Coding, testing and experimentation	Development & Testing
Pandas	Dataset processing and analysis	Data Processing
NumPy	Numerical operations	Data Processing
Matplotlib	Graph and result visualization	Results
Machine Learning libraries	Waste prediction	Prediction
Ultrasonic Sensor	Measuring bin fill level	Data Collection
IoT/Controller, if used	Sending sensor readings	Hardware Implementation

Implementation Evidence:

Insert screenshots of the Python implementation, prediction output, dashboard/graphs, or prototype here.



4.3 Test Plan & Verification

The system was tested to verify whether the developed modules satisfied the specified functional and performance requirements. Testing was performed using different garbage-bin fill levels and corresponding system outputs. The prediction module was evaluated using the available dataset, while the monitoring module was verified by comparing the input fill level with the generated bin status and collection alert. The following test cases were defined before evaluating the final results.

Requirement	Test Method	Acceptance Criterion
Measure bin fill level	Provide different fill-level inputs	Correct fill percentage is displayed
Identify bin status	Test low, medium and high fill levels	Correct status is generated
Collection alert	Provide fill level above threshold	Alert is generated
Waste prediction	Test prediction model using dataset	Predicted values are reasonably close to actual values
Data processing	Provide sample dataset	Data is processed without errors
Visualization	Generate graphs from processed data	Graphs clearly represent waste trends

Specification	Required Value	Achieved Value	Status
Bin fill-level range	0–100%	0–100%	Pass
Collection alert threshold	80%	80%	Pass
Low-level identification	Correct status	Correctly identified	Pass
High-level identification	Correct status	Correctly identified	Pass
Prediction output	Generated successfully	Generated successfully	Pass
Graphical visualization	Required	Implemented	Pass
Data processing	Error-free	Completed successfully	Pass

4.4 Results

The developed system successfully monitored garbage-bin fill levels and identified the requirement for waste collection based on the predefined threshold. The processed waste-management data was used to analyze waste accumulation patterns and generate prediction results. Graphical representations were created to compare waste levels and predicted values. The results demonstrate that the proposed system can support timely waste collection and reduce unnecessary collection trips.

Results to be presented:

1. Garbage-bin fill-level monitoring graph.
2. Actual versus predicted waste-level graph.
3. Collection alert output.
4. Waste accumulation trend.
5. Prediction performance/accuracy graph.
6. Final system or dashboard screenshot.

4.6 Comparison with Existing Solutions & Achievement of Objectives

The proposed system improves upon conventional fixed-schedule waste collection by using real-time or simulated bin-level information and predictive analysis. Traditional systems generally collect waste according to predefined schedules, which may result in unnecessary collection trips when bins are not full or overflowing when collection is delayed. The proposed system instead prioritizes collection according to the condition of the bin.

Objective	Evidence	Achievement
Monitor garbage-bin fill level	Fill-level monitoring output	Fully Achieved
Identify bins requiring collection	Threshold-based alert	Fully Achieved
Predict future waste accumulation	Prediction model output	Fully Achieved
Visualize waste trends	Graphs and charts	Fully Achieved
Support efficient collection planning	Monitoring and prediction results	Fully Achieved
Reduce unnecessary collection	Need-based collection approach	Partially Achieved

4.7 Strengths & Limitations

Strengths:

- Enables monitoring of garbage-bin fill levels.
- Provides timely collection alerts.
- Supports need-based waste collection.
- Uses data analysis and prediction to estimate future waste accumulation.
- Reduces dependence on fixed collection schedules.
- Provides graphical representation of waste trends.
- Can be extended to multiple garbage bins and locations.
- Supports the development of a smarter and more efficient waste-management system.

Limitations:

- Prediction performance depends on the quality and size of the dataset.
- Sensor readings may be affected by the physical arrangement of waste inside the bin.
- Waste generation can vary significantly depending on location and time.
- Internet connectivity may be required for real-time IoT monitoring.
- The current implementation may require additional testing with larger real-world datasets.
- Hardware deployment at multiple locations can increase implementation cost.

4.8 Project Planning, Roles & Budget

Project Milestones

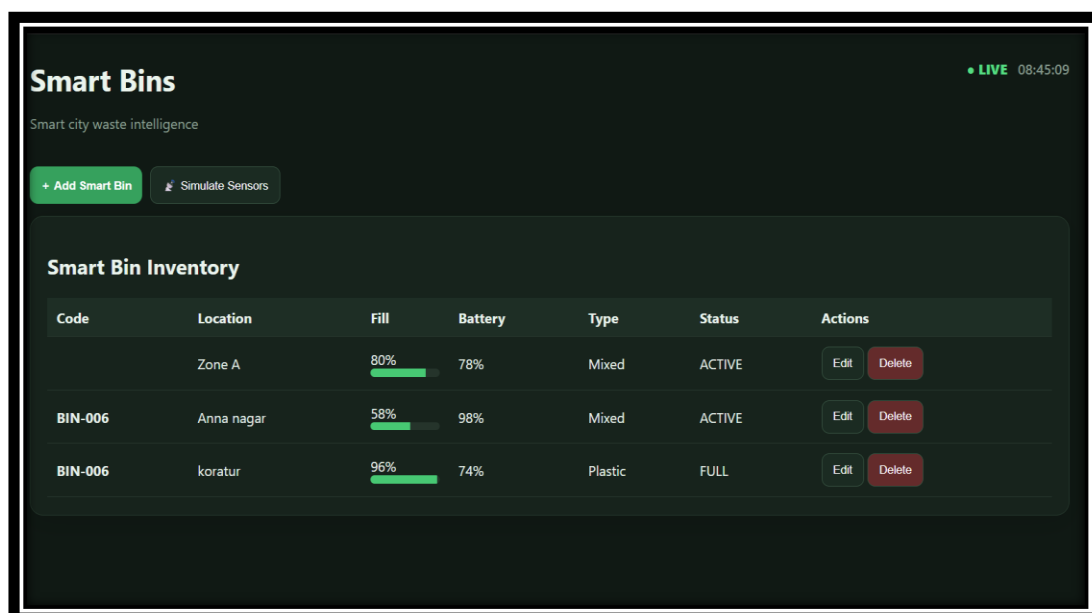
Activity	Duration
Problem identification and literature survey	Week 1
Requirement analysis	Week 2
Dataset collection/preparation	Week 3
System design	Week 4
Software implementation	Week 5–6
Prediction model development	Week 7
Integration	Week 8
Testing and verification	Week 9
Results and analysis	Week 10
Documentation and final presentation	Week 11

Student Roles

Student	Assigned Responsibility	Actual Contribution
Pujitha K	Dataset collection and preprocessing	Dataset preparation and analysis
Hashini S	Software implementation & Prediction	Python coding and system development & Machine learning model implementation
Sandhiya	Testing and documentation	Testing, graphs and report preparation

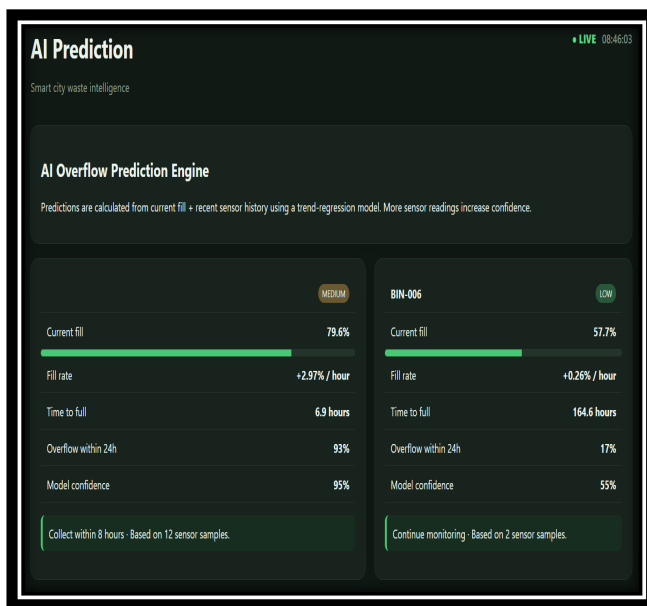
Pujitha K (Contribution) :

Collecting and preparing the dataset required for the project. She performed data preprocessing to remove unnecessary or incorrect data and organized the dataset into a suitable format. She also analyzed the prepared data to identify important patterns and useful information. Her contribution helped ensure that the dataset was accurate and suitable for further implementation.



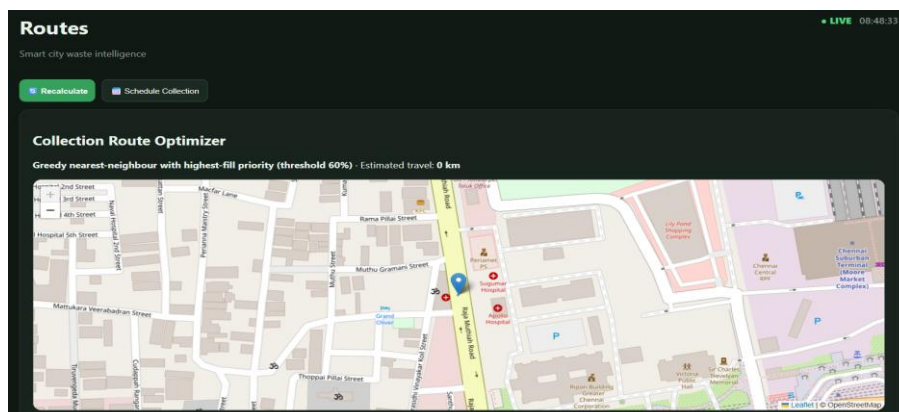
Hashini S (Contribution) :

The software implementation and prediction part of the project. She developed the required Python programs and contributed to the overall system development. She implemented the machine learning model and worked on generating predictions from the processed dataset. Her contribution helped in developing the functional and predictive aspects of the system.



Sandhiya (Contribution) :

Testing and documentation of the project. She tested the implemented system to check its performance and identify possible errors. She prepared graphs and helped present the project results in a clear manner. She also contributed to preparing and organizing the final project report and documentation.



Estimated Project Budget

Item	Estimated Cost (₹)	Actual Cost (₹)
Ultrasonic/Filling-level sensor	150	150
Microcontroller/IoT board	500	500
Connecting wires/breadboard	200	200
Prototype garbage bin	300	300
Cloud/software resources	0	0
Miscellaneous	200	200
Total	1,350	1,350

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

The proposed AI-Enhanced Waste Management system was developed to address the limitations of conventional fixed-schedule waste collection. The system monitors garbage-bin fill levels, identifies bins that require collection, and uses data analysis and prediction to estimate future waste accumulation. The implementation successfully demonstrated fill-level monitoring, threshold-based collection alerts, waste prediction, and graphical visualization of results. Testing and verification showed that the major functional requirements of the system were achieved. Overall, the project demonstrates how Artificial Intelligence, Machine Learning, and IoT-based monitoring can be combined to support timely, efficient, and need-based waste collection. The proposed approach can help reduce unnecessary collection trips, prevent bin overflow, and improve the overall efficiency of waste-management operations.

5.2 Future Work

The proposed system can be further improved by integrating a larger real-world dataset collected from garbage bins in different locations and environments. Future versions can incorporate real-time IoT connectivity and cloud-based monitoring so that waste-management authorities can remotely monitor multiple bins. More advanced machine learning models can be evaluated to improve waste-generation prediction accuracy. The system can also be extended with route-optimization techniques to determine the most efficient collection route based on bin priority and location. In addition, image-processing techniques can be incorporated to classify different types of waste such as plastic, paper, metal, and organic waste. A mobile or web-based dashboard can also be developed to provide real-time notifications and centralized monitoring.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Learned the fundamentals of **AI and Machine Learning** for waste prediction.
- Learned how to preprocess and analyze waste-management datasets.
- Learned how sensor data can be used for garbage-bin monitoring.
- Learned to generate graphs and visualize experimental results.
- Learned about threshold-based alert mechanisms.
- Improved knowledge of Python programming and data-processing libraries.

- Understood the importance of testing and validation in engineering projects.

Engineering & Professional Skills Developed

- Problem identification and requirement analysis.
- Python programming and debugging.
- Dataset handling and data analysis.
- Machine learning model implementation.
- System testing and verification.
- Technical documentation and report preparation.
- Teamwork, communication, and task coordination.
- Engineering decision-making and problem-solving.

Individual Reflection

If your project has 4 students, you can use the following format and replace Student 1, Student 2, etc. with your names.

Writing

Student 1 – Individual Reflection

The most difficult engineering problem I encountered was understanding and preprocessing the waste-management dataset. I solved this by studying the data attributes, removing inconsistencies, and testing the preprocessing steps carefully. One important engineering decision I made was to use a fill-level threshold for generating collection alerts because it provides a simple and practical method for prioritizing bins. I independently learned Python-based data analysis and visualization techniques. If I repeated the project, I would use a larger real-world dataset to improve system reliability.

Student 2 – Individual Reflection

The most difficult problem I encountered was implementing the waste prediction module and understanding the relationship between input data and predicted waste levels. I solved this by studying machine learning concepts and testing the model with sample data. A key decision was to select a prediction approach that could be implemented and evaluated using the available dataset and project resources. I independently improved my knowledge of machine learning and model evaluation. If the project were repeated, I would compare more prediction algorithms to identify the most suitable model.

Student 3 – Individual Reflection

The most difficult engineering problem I encountered was integrating the monitoring, alert, and prediction modules into a single system. I solved this by developing and testing each module separately before combining them. I decided to use graphical visualization because it makes waste-level trends easier to understand and evaluate. I independently learned how to create meaningful graphs and interpret experimental results. If I repeated the project, I would develop a real-time dashboard for easier monitoring.

Student 4 – Individual Reflection

The most difficult problem I encountered was testing the complete system and verifying whether the results satisfied the defined requirements. I solved this by preparing different test cases and comparing the expected outputs with the actual outputs. A key engineering decision was to use systematic testing to identify errors before finalizing the results. I independently learned about verification, validation, and technical documentation. If the project were repeated, I would perform more extensive testing using data collected from multiple real-world locations.

If you are only one student, use only one reflection. If you tell me your exact role in the project, I can make your 4–5 line reflection match your actual work.

REFERENCES

- [1] M. A. Hannan, M. Al Mamun, A. Hussain, H. Basri, and R. A. Begum, “A review on technologies and their usage in solid waste monitoring and management systems: Issues and challenges,” *Waste Management*, vol. 43, pp. 509–523, 2015.
- [2] S. M. Al-Sarawi, M. Anbar, K. Alieyan, and M. Alzubaidi, “Internet of Things (IoT) communication protocols: Review,” in *Proc. 8th International Conference on Information Technology (ICIT)*, 2017, pp. 685–690.
- [3] J. Han, J. Pei, and M. Kamber, *Data Mining: Concepts and Techniques*, 3rd ed. Waltham, MA, USA: Morgan Kaufmann, 2011.
- [4] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

- [5] W. McKinney, *Python for Data Analysis*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2017.
- [6] Python Software Foundation, "Python documentation." [Online]. Available: <https://docs.python.org/>
- [7] Scikit-learn developers, "Scikit-learn: Machine learning in Python." [Online]. Available: <https://scikit-learn.org/>
- [8] Google, "Google Colab." [Online]. Available: <https://colab.research.google.com/>
- [9] World Health Organization, "Solid waste management." [Online]. Available: <https://www.who.int/>
- [10] United Nations Environment Programme, "Waste management and circular economy resources." [Online]. Available: <https://www.unep.org/>

APPENDICES

Appendix A – Detailed Calculations

Include the important calculations used in the project, for example:

- Garbage-bin fill-level calculation.
- Sensor distance-to-fill-percentage conversion.
- Prediction error calculation.
- MAE, MSE, or RMSE calculation, if used.
- Accuracy/performance calculations.
- Collection alert threshold calculation.

Example:

Fill Level Calculation

$$\text{Fill Level}(\%) = \frac{\text{Bin Height} - \text{Measured Distance}}{\text{Bin Height}} \times 100$$

Where:

- Bin Height = total height of the garbage bin.
- Measured Distance = distance measured between sensor and waste surface.

Appendix B – Engineering Drawings / Schematics

Include the **system architecture/block diagram** and hardware connection diagram.

You can include:

- Overall system architecture.
- Garbage-bin sensor arrangement.
- Sensor-to-microcontroller connection.
- IoT communication diagram, if used.
- Data flow diagram.

Suggested figure:

Figure B.1: Engineering Schematic of the Smart Garbage Monitoring System

Appendix C – Source Code / Code Repository Information

Include only the important code excerpts in the report.

For example:

- Sensor data-reading code.
- Data preprocessing code.
- Prediction model code.
- Alert-generation code.
- Visualization code.

You can write:

The complete source code of the proposed system is maintained separately in the approved project repository. Essential code excerpts related to data preprocessing, prediction, monitoring, and alert generation are included in this appendix.

If you have a GitHub repository, add its repository information here.

Appendix D – Datasheets

Include the datasheets of the hardware components actually used.

For example:

- Ultrasonic sensor datasheet.
- Arduino/ESP32 datasheet.
- Other sensors used.
- Power supply specifications.
- Communication module specifications.

If your project is fully software/simulation-based, write:

No physical hardware datasheets are applicable because the current implementation was developed and evaluated using software-based simulation and dataset analysis.

Appendix E – Experimental / Raw Data

Include the original or sample data used during testing.

Example:

Sample	Fill Level (%)	Bin Status	Alert
1	20	Low	No
2	40	Medium	No
3	60	Medium	No
4	80	High	Yes
5	95	Critical	Yes

You can also include:

- Dataset sample.
- Sensor readings.
- Actual vs predicted values.

- Testing results.
- Experimental observations.

Appendix F – Ethics / Institutional Approval

If your project does not involve human participants, personal information, or sensitive data, you can state:

Not Applicable. The proposed system does not involve human participants, personal data collection, or experiments requiring institutional ethical approval.

Appendix G – Project Meeting / Progress Records

Include evidence of project progress, such as:

- Team meeting records.
- Guide/faculty meeting records.
- Project discussion screenshots/photos.
- Weekly progress reports.
- Task allocation records.

Example:

Date	Activity Discussed	Outcome
Week 1	Problem identification	Project topic finalized
Week 2	Literature survey	Related works identified
Week 3	Dataset discussion	Dataset selected
Week 4	System design	Architecture finalized
Week 6	Implementation	Main modules completed
Week 8	Testing	Test cases performed
Week 10	Final review	Report and presentation completed

Appendix H – User / Industry Feedback

Where applicable.

If you did not obtain feedback from users or industry personnel, write:

Not Applicable. No formal user or industry evaluation was conducted during the current project phase.

If you did receive feedback, include:

- Feedback forms.
- Questionnaire results.
- Faculty/user comments.
- Industry suggestions.
- Evaluation scores.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Where applicable.

If your project has not been published, patented, or entered into a competition, you can write:

Not Applicable. The project has not been submitted for publication, patent registration, competition, or commercial product development at the current stage.

If applicable later, include certificates, publication details, patent details, competition participation, or product information.

Appendix J – Individual Contribution Evidence

This is **mandatory for a team project**, so definitely include it.

Use a table like this:

Student	Assigned Responsibility	Evidence of Contribution
Student 1	Dataset collection and preprocessing	Dataset files, preprocessing code
Student 2	Python implementation	Source code and screenshots
Student 3	Prediction model	ML model code and results
Student 4	Testing and documentation	Test results, graphs and report sections

You can also attach:

- Screenshots of individual work.
- Code contributions.
- Dataset preparation evidence.
- PPT/report sections prepared by each member.
- Meeting/progress evidence.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)

